

Experiment 3: Survey Report on Emerging Trends in Low-Code Application Development

1. AIM

To study the emerging trends in **low-code application development** and analyse how low-code platforms such as **Zoho Creator** can be used to develop a **Vehicle Service & Repair Request Management** application with minimal traditional programming.

2. Traditional Application Development vs Emerging Trends in Low-Code Application Development

Traditional Application Development	Low-Code Application Development
Requires extensive programming knowledge.	Requires minimal coding knowledge.
Developers manually write most of the application code.	Applications are built using visual drag-and-drop components.
Development takes more time.	Applications can be developed faster.
Requires separate development for UI, database and workflows.	Forms, reports, databases and workflows can be configured in one platform.
Testing and debugging require more development effort.	Many validations and features are available as built-in components.
Changes usually require code modifications.	Changes can often be made through visual configuration.
Suitable for highly customized and complex applications.	Suitable for business applications and rapid application development.
Requires dedicated developers for maintenance.	Business users and developers can collaboratively maintain applications.

Emerging Trends in Low-Code Development

- Drag-and-drop application builders
 - Visual workflow automation
 - Cloud-based application development
 - AI-assisted application development
 - Integration with APIs and external services
 - Reusable components and templates
 - Rapid development and deployment
 - Mobile-friendly application development
 - Low-code database and report generation
 - Citizen development, where non-professional programmers can build applications.
-

3. Benefits and Challenges in Low-Code

Benefits

1. **Faster Development** - Applications can be developed quickly using visual components.
2. **Less Coding** - Reduces the amount of traditional programming required.
3. **Easy to Use** - Drag-and-drop interfaces make development easier for beginners.
4. **Easy Maintenance** - Forms, fields and workflows can be modified easily.
5. **Automation** - Notifications, emails and workflows can be automated.
6. **Reduced Development Cost** - Less development time can reduce overall cost.
7. **Rapid Prototyping** - Applications can be tested and modified quickly.
8. **Cloud Accessibility** - Applications can be accessed through the internet from different devices.

Challenges

1. **Platform Dependency** - Applications may become dependent on a particular low-code platform.
2. **Limited Customization** - Highly complex requirements may require additional coding.
3. **Security Concerns** - Applications must be configured properly to protect user data.
4. **Scalability Issues** - Very large or complex applications may require additional resources.
5. **Integration Limitations** - Some external systems may require custom APIs or code.
6. **Vendor Lock-in** - Moving an application to another platform can be difficult.
7. **Learning Platform-Specific Features** - Users still need to understand the platform's tools and limitations.

4. Comparison of Existing Low-Code Platforms

Feature	Zoho Creator	Microsoft Power Apps	Google AppSheet	Mendix
Development Style	Low-code / visual	Low-code / visual	No-code / low-code	Low-code
Form Builder	Yes	Yes	Yes	Yes
Database Support	Built-in	Dataverse	Built-in / external sources	Built-in
Workflow Automation	Yes	Power Automate	Automation Bots	Workflow/Microflows
Reports & Dashboards	Yes	Yes	Yes	Yes
Mobile Application	Yes	Yes	Yes	Yes
AI Features	Available	Available	Available	Available

Feature	Zoho Creator	Microsoft Power Apps	Google AppSheet	Mendix
API Integration	Yes	Yes	Yes	Yes
Suitable For	Business applications	Enterprise applications	Data-driven applications	Enterprise applications
Main Advantage	Easy application and workflow development	Strong Microsoft ecosystem integration	Simple and fast app creation	Enterprise-level application development

Application Used in This Experiment

Platform: Zoho Creator

Application: Vehicle Service & Repair Request Management

Form: Service_Request_Form

Report: Service_Request_Report

The application uses the **13 Basic Fields** in Zoho Creator to collect owner details, vehicle information, contact details, service requirements, scheduling information and repair authorization.

The screenshot displays the Zoho Creator mobile application interface. On the left is a dark sidebar menu with the title 'My first Creator app' and a lightning bolt icon. The menu includes options: 'Home', 'All Requests', 'Add Request', 'Vehicle Service Requests' (expanded), 'Vehicle Service Request Form' (highlighted in blue), and 'All Vehicle Service Requests'. At the bottom of the sidebar is the user's name 'Abirami'. The main content area is titled 'Vehicle Service Request Form' and contains the following fields:

- Owner_Name:** Split into 'First Name' and 'Last Name' text input fields.
- Email_Address:** A single text input field.
- Pickup_Address:** Split into 'Address Line 1' and 'Address Line 2' text input fields.
- City / District:** A text input field.
- State / Province:** A dropdown menu with '-Select-' as the placeholder.
- Postal Code:** A text input field.
- Country:** A dropdown menu with a flag icon and '+91' as the selected value.
- Contact_Number:** A text input field containing the number '81234 56789'.
- Vehicle_Registration_Number:** A text input field.
- Issue_Description:** A large text area for input.

The screenshot shows a mobile application interface for a 'Vehicle Service Request Form'. The app is titled 'My first Creator app'. The left sidebar contains navigation options: Home, All Requests, Add Request, Vehicle Service Requests (expanded), Vehicle Service Request Form (selected), and All Vehicle Service Requests. The main form area contains the following fields:

- Issue_Description: A large text input field.
- Odometer_Reading: A text input field with a placeholder '#####'.
- Preferred_Service_Date: A date input field with a placeholder 'dd-MMM-yyyy'.
- Preferred_Dropoff_Time: A time input field with a placeholder 'HH:mm:ss'.
- Vehicle_Type: A dropdown menu with '-Select-' as the current selection.
- Service_Type: A radio button group with three options: 'Routine Service', 'Breakdown Repair', and 'Accident Repair'.
- Additional_Services_Requested: A dropdown menu with '-Select-' as the current selection.
- Authorize_Estimate_And_Repair: A checkbox labeled 'Agreed'.

A blue 'Submit' button is located at the bottom right of the form.

5. RESULT

The survey on **emerging trends in low-code application development** was successfully completed. The **Vehicle Service & Repair Request Management** application was designed using **Zoho Creator** by utilizing the available Basic Fields, form validations, report generation and workflow features. The experiment demonstrated how low-code platforms can reduce development effort and support the rapid creation of real-world business applications such as vehicle service scheduling and request management.